CSED211 컴퓨터 시스템 2026 가을 · 중간고사 10/29 (목) ·

Data Lab: bitwise 연산만으로 함수 구현하기

DL1·DL2 규칙과 비트/부동소수점 접근법 정리 · 연습 문제는 문제 풀이 페이지에

1 Lab의 규칙: 제약을 지키는 것 자체가 문제

- bits.c 안의 함수들을 bitwise operation만으로 채우는 lab
- 세 가지 제약: ① 허용된 operation만 사용 ② 정해진 최대 operation 개수 이하 ③ programming rule 준수. 위반 시 해당 문제 0점
- 검사 도구: `btest` (함수가 올바른 값을 내는지 테스트) · `dlc` (규칙 위반 검사)

정수 문제 (DL1 전체 + DL2 Q1~Q2): 허용 / 금지

- 허용: 정수 상수 0~255 (0xFF) · 인자와 지역변수 · unary ! ~ · binary & ^ | + << >> (문제마다 허용 목록과 개수 상한이 다름) · int type
- 금지: if · do · while · for · switch · macro · 다른 함수 호출 · && · || · – 등 허용되지 않은 연산 · int 외의 type (array, struct, union)
- 상수가 0~255뿐 → 0xFFFFFFFF, 0x80000000 같은 큰 상수는 직접 못 씀 → `~0`, `1 << 31`처럼 연산으로 만들기
- -가 없음 → 음수화는 two's complement 성질 $-x = \sim x + 1$로

부동소수점 문제 (DL2 Q3~Q6): 규칙이 다름

- float는 bit pattern을 담은 unsigned로 전달·반환
- 허용: 모든 int/unsigned 연산, || &&, if, while, 임의 크기 상수
- 금지: union, struct, array, float 자료형·연산·상수
- 가정: 32-bit two's complement · right shift는 arithmetic · word size 이상 shift는 동작 예측 불가

2 비트 기본기 복습

연습 문제는 문제 풀이 페이지로 옮김

- 비트 기본기 연습 8문제 (힌트·정답 토글) → 문제 풀이의 Data Lab 연습 섹션

Byte와 진법

- Byte = 8 bits · hexadecimal은 base-16 (0~9, A~F) → **hex 한 자리 = 4 bit**
- hex → binary는 자리마다 4 bit로 바꾸면 끝: b = 1011, 9 = 1001

손계산 1 · int a = 0xb9602353의 bit 배치

```
Byte 3: 0xb9 = 1011 1001   ← MSB (왼쪽 끝)
Byte 2: 0x60 = 0110 0000
Byte 1: 0x23 = 0010 0011
Byte 0: 0x53 = 0101 0011   ← LSB (오른쪽 끝)
MSB = 1 → signed int로 보면 음수
```

Bitwise vs Logical operator: 혼동 금지

- bitwise (~ & | ^)는 bit pattern의 각 bit에 적용 / logical (&& || !)는 결과가 항상 0 또는 1

> •예: bitwise 0x69 & 0x55 = 0x41 이지만 logical 0x69 && 0x55 = 1 · ~0x55 = 0xAA (8-bit) 이지만 !0x55 = 0

- assignment 버전: &= |= ^= <<= >>= (a &= b → a = a & b)

A	B	A & B	A | B	A ^ B	~A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

- AND: 둘 다 1일 때 1 · OR: 하나라도 1이면 1 · XOR: 둘 중 하나만 1일 때 1 · NOT: 0 ↔ 1
- bit vector에 bitwise로 적용 → Boolean algebra의 성질이 모두 성립 (De Morgan 포함)

손계산 2 · bit vector 연산 (01101001과 01010101)

```
  01101001    01101001    01101001    ~ 01010101
& 01010101  | 01010101  ^ 01010101  ──────────
──────────  ──────────  ──────────    10101010
  01000001    01111101    00111100
= 0x41        = 0x7D        = 0x3C
```

De Morgan: lab에서 직접 쓰게 됨

$$\sim(a \,\&\, b) = \sim a \mid \sim b \qquad \sim(a \mid b) = \sim a \,\&\, \sim b$$

Shift

- left shift x << y: 왼쪽으로 y칸, 오른쪽은 0으로 채움. 위로 밀려난 bit는 사라짐
- right shift x >> y: logical은 왼쪽을 0으로, arithmetic은 sign bit로 채움
- 오른쪽 시프트 x >> n: unsigned 값은 logical right shift로 처리하며, signed 값은 이 Data Lab의 명시적 가정에 따라 arithmetic right shift로 처리한다. Arithmetic shift는 왼쪽을 부호 비트로 채우고 logical shift는 왼쪽을 0으로 채운다. 단, ISO C에서 음수인 signed 정수의 오른쪽 시프트 결과는 구현 정의이므로, signed >>가 항상 arithmetic이라고 C 언어 전체의 보장으로 일반화해서는 안 된다.

a = 01100010 (MSB 0) · b = 10100010 (MSB 1) — 8-bit

x << 3	a → 00010000 · b → 00010000 (위로 밀린 3 bit 소실, 아래는 0)
x >> 2 logical	a → 00011000 · b → 00101000 (왼쪽을 0으로 채움)
x >> 2 arithmetic	a → 00011000 · b → 11101000 (왼쪽을 sign bit로 채움)

**a는 logical = arithmetic이지만 b는 MSB = 1이라 결과가 다름 →
부호 있는 값의 >>는 항상 arithmetic이라고 가정하고 풀 것**

손계산 3: 같은 bit pattern도 type에 따라 shift 결과가 갈림

3 Data Lab 1 문제 풀이 (정수 6개)

Q0 minusOne (수업 예제): -1 반환 · 허용 ! ~ & ^ | + << >> · 최대 2 ops

- 허용 연산에 -가 없으니 return -1;은 규칙 위반 → -1의 bit pattern(모든 bit 1)은 0을 반전한 것

```
int minusOne(void) { return ~0; }  /* 1 op */
```

허용된 연산의 조합으로 원하는 bit pattern을 만드는 것이 lab의 기본 패턴.

Q1 bitNor(x, y): ~(x|y) · 허용 ~ & · 최대 8 ops

- 허용 연산에 |가 없음 → De Morgan으로 |를 ~와 &로 바꿈

```
int bitNor(int x, int y) { return ~x & ~y; }  /* 3 ops */
```

Q2 isZero(x): x가 0이면 1 · 최대 2 ops

- !x는 x가 0이면 1, 아니면 0. 정의와 정확히 일치. bitwise ~가 아니라 logical !를 써야 함 (~0은 -1)

```
int isZero(int x) { return !x; }  /* 1 op */
```

Q3 addOK(x, y): x+y가 overflow 없이 계산되면 1 · 최대 20 ops

- signed overflow는 x와 y의 부호가 같은데 합 s의 부호가 다를 때만 발생. 부호가 다른 두 수의 합은 절대 overflow 안 됨
- x ^ s의 MSB = 1 ⇔ x와 s 부호가 다름 → (x ^ s) & (y ^ s)의 MSB = 1 ⇔ overflow
- >> 31은 signed라 arithmetic → 결과는 0 또는 -1 → !로 1/0 변환

```
int addOK(int x, int y) {
  int s = x + y;
  return !(((x ^ s) & (y ^ s)) >> 31);  /* + ^ ^ & >> ! : 6 ops */
}
```

손계산 4 — addOK의 overflow 판정

- 사례 1: 양의 최댓값에 1을 더하는 경우. x=0x7FFFFFFF, y=1. s=0x80000000. (x^s)&(y^s)=0x80000001. MSB가 1이므로 overflow가 발생한다. 반환값은 0이다.
- 사례 2: 최솟값에 −1을 더하는 경우. x=TMin, y=−1. 32-bit bit-vector 모델에서 s=0x7FFFFFFF이다. (x^s)&(y^s)=0x80000000. MSB가 1이므로 overflow가 발생한다. 반환값은 0이다.
- 사례 3: 부호가 다른 두 수를 더하는 경우. x=100, y=−200. s=−100=0xFFFFFF9C. (x^s)&(y^s)=0x000000A0. MSB가 0이므로 overflow가 발생하지 않는다. 반환값은 1이다.
- 사례 4: 두 양수를 더해 음수 비트 패턴이 되는 경우. x=y=0x40000000. s=0x80000000. (x^s)&(y^s)=0xC0000000. MSB가 1이므로 overflow가 발생한다. 반환값은 0이다.

Q4 absVal(x): 절댓값 · 최대 10 ops

- m = x >> 31 (arithmetic) → x ≥ 0이면 0, x < 0이면 -1 (모든 bit 1). 분기 없이 부호에 따라 연산을 고르는 sign mask 기법
- m = 0: (x + 0) ^ 0 = x · m = -1: (x − 1) ^ (전부 1) = ~(x − 1) = −x

```
int absVal(int x) {
  int m = x >> 31;
  return (x + m) ^ m;  /* >> + ^ : 3 ops */
}
```

손계산 5: x = 5 → m = 0 → 5 · x = -5 → m = 0xFFFFFFFF, x+m = -6, ^ m → 5 · x = TMin → 절댓값 2^{31}은 int로 표현 불가 → TMin 그대로 (Data Lab 비트벡터 모델에서 -TMin = TMin — ISO C에서는 signed overflow UB)

Q5 logicalShift(x, n): n칸 logical right shift · 최대 20 ops

- signed int의 >>는 arithmetic → x가 음수면 왼쪽이 1로 채워짐 → shift 후 위쪽 n bit를 0으로 만드는 mask를 AND
- mask: (1 << 31) >> n (arithmetic)이면 위쪽 n+1 bit가 1 → 한 칸 더 밀어(<< 1) 위쪽 n bit만 1로 만든 뒤 ~
- >> (n+1)로 한 번에 밀지 않는 이유: n = 31이면 shift 양이 32 → C에서 정의되지 않은 동작
- n = 0이면 (0x80000000 >> 0) << 1 = 0 → mask = ~0 = 전부 1 → 그대로 통과

```
int logicalShift(int x, int n) {
  int mask = ~(((1 << 31) >> n) << 1);
  return (x >> n) & mask;  /* 6 ops */
}
```

손계산 6: logicalShift(-16, 2): x = 0xFFFFFFF0 → x >> 2 = 0xFFFFFFFC · (1<<31) >> 2 = 0xE0000000 → << 1 = 0xC0000000 → mask = 0x3FFFFFFF · 결과 0xFFFFFFFC & 0x3FFFFFFF = 0x3FFFFFFC (unsigned 계산과 일치)

4 Data Lab 2: FP 문제 접근

문제 목록

- 정수 (DL1과 같은 규칙): Q1 negate(x): 최대 5 ops · Q2 isLess(x, y): 최대 24 ops
- 부동소수점 (별도 규칙, 섹션 1): Q3 float_abs(uf): 최대 10 · Q4 float_twice(uf): 최대 30 · Q5 float_i2f(x): 최대 30 · Q6 float_f2i(uf): 최대 30

FP 문제 공통 접근 (보충): uf를 세 field로 분리해 놓고 생각

```
sign = uf >> 31;  exp = (uf >> 23) & 0xFF;  frac = uf & 0x7FFFFF;
```

- 먼저 exp == 0xFF (NaN/∞), exp == 0 (denorm/0), 그 외 (normalized)로 경우를 나눔
- float_abs: exp = 0xFF이고 frac != 0이면 NaN이므로 입력 비트 패턴 uf를 그대로 반환한다. 그 외에는 uf & 0x7FFFFFFF로 sign bit를 0으로 만들어 절댓값을 반환한다. 따라서 유한값과 ±∞는 부호 비트만 제거하고, NaN은 원래 비트 패턴을 보존한다.
- twice: normalized는 exp + 1 (결과가 0xFF면 ∞) · denorm은 (exp|frac) 비트를 통째로 한 칸 왼쪽으로 (frac 최상위 bit의 carry가 exp로 넘어가 자연스럽게 normalized로 — exp를 따로 갱신할 필요 없음) · NaN/∞는 그대로
- i2f: 0과 TMin 특별 취급 · 절댓값의 최상위 1 위치로 E 결정 · 23 bit 넘는 부분은 round-to-even
- f2i: E < 0이면 0 · int 범위 초과·NaN·∞이면 0x80000000 · 그 외는 1.f를 E만큼 shift 후 sign 적용

Two's complement 복습 (자세한 건 Lec2)

범위와 inversion

$\mathrm{TMin} = -2^{w-1}\ (10\ldots0)$ $\mathrm{TMax} = 2^{w-1}-1\ (01\ldots1)$ $-x = \sim x + 1$

$-\mathrm{TMin} = \mathrm{TMin}$ (양수 쪽에 대응 값이 없음) · 4-bit 예: 1000 = −8, 1011 = −5, 1111 = −1

- TAdd와 UAdd의 비트 수준 동작: w비트 bit-vector 모델에서 signed 덧셈과 unsigned 덧셈은 하위 w 비트가 동일하며, 결과는 (u+v) mod 2^w로 해석할 수 있다. 이때 최상위 비트 밖으로 나간 carry는 버려진다. ISO C의 주의점: signed 정수의 u+v가 표현 범위를 벗어나면 undefined behavior가 발생한다. 또한 (int)((unsigned)u + (unsigned)v)에서 unsigned 결과가 int의 표현 범위를 벗어나면 변환 결과는 구현 정의일 수 있다. 따라서 두 C 식의 값이 항상 같다고 단정해서는 안 된다. Data Lab에서의 wraparound 계산은 과제에서 가정한 32-bit two's-complement bit-vector 모델과 해당 실행 환경을 기준으로 해석한다.

IEEE 754 복습 (자세한 건 Lec3)

- 형식 $V = (-1)^s \times M \times 2^E$ · s: sign bit · exp field e: E를 encoding (E와 같지 않음) · frac field f: M을 encoding (M과 같지 않음)

- IEEE 754 형식: single precision은 부호 1비트, 지수 8비트, 소수부 23비트, Bias 127이다. Double precision은 부호 1비트, 지수 11비트, 소수부 52비트, Bias 1023이다. Intel x87 80-bit extended 형식은 부호 1비트, 지수 15비트, 명시적 정수 비트 1개와 소수부 63비트로 이루어진 64비트 유효숫자를 사용하며, Bias는 16383이다.
- normalized: Single과 double의 normalized 수는 지수 필드가 전부 0이거나 전부 1인 경우가 아닐 때 E = exp − Bias, M = 1.f로 표현한다. 이 두 형식에서는 선행 1을 저장하지 않는다(implied leading 1). Intel x87 80-bit extended 형식에서는 정규화된 유효숫자의 정수 비트가 명시적으로 저장되므로, 선행 1을 저장하지 않는다는 설명을 그대로 적용해서는 안 된다.
- denormalized: exp = 00⋯0 → E = 1 − Bias, M = 0.f → 0 (f = 0, +0/−0 구분)과 0에 아주 가까운 수 (gradual underflow)
- special: exp = 11⋯1 → f = 0이면 ±∞ (1.0/0.0), f ≠ 0이면 NaN (sqrt(−1), ∞−∞, ∞×0)
- denorm의 E가 −Bias가 아니라 1 − Bias인 이유: 가장 큰 denorm과 가장 작은 normalized 사이가 끊김 없이 이어지게 하려고

5 손계산 모음

15213.0을 single precision으로 (0x466DB400)

0	1000 1100	110 1101 1011 0100 0000 0000
s	exp (8) = 13 + 127 = 140	frac (23) = 1.1101101101101의 소수점 뒤 13 bit + 0 10개

```
4 bit씩: 0100 0110 0110 1101 1011 0100 0000 0000 = 0x466DB400
```

$$15213.0 = 1.1101101101101_2 \times 2^{13}$$

Tiny float (s 1, exp 4 bit bias 7, frac 3 bit): IEEE와 같은 규칙

s exp frac	E	값	의미
0 0000 000	−6	0	zero
0 0000 001	−6	1/8 × 1/64 = 1/512	0에 가장 가까운 수
0 0000 111	−6	7/8 × 1/64 = 7/512	가장 큰 denorm

s exp frac	E	값	의미
0 0001 000	−6	8/8 × 1/64 = 8/512	가장 작은 normalized
0 0110 111	−1	15/8 × 1/2 = 15/16	1보다 작은 수 중 가장 가까운 수
0 0111 000	0	1	없음
0 0111 001	0	9/8	1보다 큰 수 중 가장 가까운 수
0 1110 111	7	15/8 × 128 = 240	가장 큰 normalized
0 1111 000	없음	+∞	special

- 가장 큰 denorm 7/512 → 가장 작은 normalized 8/512: 1/512 간격으로 매끄럽게 이어짐 (denorm E = 1 − Bias 덕분)
- 슬라이드 표의 0 0111 110 / 0 0111 111 (E = 7, 224·240) 행은 exp가 1110이어야 맞음 (E = 14 − 7 = 7). 원본 노트의 교정 메모

6 시험 포인트

Data Lab에서 시험에 나올 수 있는 것

- bitwise (~ & | ^)는 bit마다, logical (! && ||)은 결과가 항상 0/1
- C의 >>: unsigned → logical, signed → arithmetic (Data Lab의 가정에 따른 처리이며, ISO C에서 음수인 signed 정수의 오른쪽 시프트는 구현 정의). 8-bit 예: 10100010 >> 2 → logical 00101000, arithmetic 11101000
- De Morgan → bitNor의 핵심
- 규칙: 상수 0~255, 제어문·macro·함수 호출·int 외 type 금지, -와 && || 금지 (정수 문제)
- −1 = ~0 · TMin = 1 << 31 · $-x = \sim x + 1$
- overflow 판정: 같은 부호끼리 더했는데 결과 부호가 다를 때
- sign mask x >> 31 (0 또는 −1)로 분기 없이 연산 선택 · TMin의 절댓값은 표현 불가 → TMin

7 연습문제

연습 문제는 문제 풀이 페이지의 Data Lab 연습 섹션으로 옮김 (비트 기본기 8문제, 힌트·정답 토글).

8 작업 환경 (Linux 기본 명령)

- ls (-a) 디렉토리 목록 (숨김 포함) · cd [dir] 이동 (. 현재, .. 상위) · cp [src] [new] 복사 · mv 이름 변경 (이동) · mkdir 디렉토리 생성 · rm 삭제 (되돌릴 수 없음, 디렉토리는 -rf)
- gcc [files]는 컴파일하여 기본 실행 파일 a.out을 생성한다 · gcc -o [out] [files]는 출력 실행 파일의 이름을 [out]으로 지정한다 · ./[exe] 실행 · chmod +x 실행 권한 · tar -xvf [archive.tar]는 tar 아카이브를 해제한다 · tar -cvf [archive.tar] [files]는 파일들을 압축하지 않고 tar 아카이브로 묶는다 · tar -czvf [archive.tar.gz] [files]는 파일들을 gzip으로 압축하여 tar 아카이브를 생성한다 · cat 내용 출력 · scp -P 2022 원격 복사 (port는 대문자 P)
- editor: vim (vi [file] → a로 편집 → esc, :wq 저장·종료), VSCode · Emacs · Nano